

CareerCompass AI Engine: Global Skill NER training (Autonomous)

This notebook trains a custom NER model to extract skills and roles.

Strategy: We utilize a **Synthetic Data Augmentation** engine to bootstrap our training. This avoids dependencies on private/gated external datasets and ensures 100% reliability for the graduation project.

Step 1: Environment Setup & Dependencies Installation

```
# امسح الكود القديم وحط ده مكانه في أول خلية
!pip uninstall torch torchvision torchaudio -y
!pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu121
!pip install transformers datasets seqeval evaluate accelerate -U
```

```
Found existing installation: torch 2.10.0+cu128
Uninstalling torch-2.10.0+cu128:
  Successfully uninstalled torch-2.10.0+cu128
Found existing installation: torchvision 0.25.0+cu128
Uninstalling torchvision-0.25.0+cu128:
  Successfully uninstalled torchvision-0.25.0+cu128
Found existing installation: torchaudio 2.10.0+cu128
Uninstalling torchaudio-2.10.0+cu128:
  Successfully uninstalled torchaudio-2.10.0+cu128
Looking in indexes: https://download.pytorch.org/whl/cu121
Collecting torch
  Downloading https://download.pytorch.org/whl/cu121/torch-2.5.1%2Bcu121-cp312-cp312-linux_x86_64.whl (780.4 MB)
    780.4/780.4 MB 1.7 MB/s eta 0:00:00
Collecting torchvision
  Downloading https://download-r2.pytorch.org/whl/cu121/torchvision-0.20.1%2Bcu121-cp312-cp312-linux_x86_64.whl (7.3 MB)
    7.3/7.3 MB 69.6 MB/s eta 0:00:00
Collecting torchaudio
  Downloading https://download-r2.pytorch.org/whl/cu121/torchaudio-2.5.1%2Bcu121-cp312-cp312-linux_x86_64.whl (3.4 MB)
    3.4/3.4 MB 90.9 MB/s eta 0:00:00
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch) (3.25.2)
Requirement already satisfied: typing-extensions>=4.8.0 in /usr/local/lib/python3.12/dist-packages (from torch) (4.15.0)
Requirement already satisfied: networkx in /usr/local/lib/python3.12/dist-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from torch) (2025.3.0)
Collecting nvidia-cuda-nvrtc-cu12==12.1.105 (from torch)
  Downloading https://download.pytorch.org/whl/cu121/nvidia_cuda_nvrtc_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (23.7 MB)
    23.7/23.7 MB 65.1 MB/s eta 0:00:00
Collecting nvidia-cuda-runtime-cu12==12.1.105 (from torch)
  Downloading https://download.pytorch.org/whl/cu121/nvidia_cuda_runtime_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (823 kB)
    823.6/823.6 kB 64.6 MB/s eta 0:00:00
Collecting nvidia-cuda-cupti-cu12==12.1.105 (from torch)
  Downloading https://download.pytorch.org/whl/cu121/nvidia_cuda_cupti_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (14.1 MB)
    14.1/14.1 MB 83.8 MB/s eta 0:00:00
Collecting nvidia-cudnn-cu12==9.1.0.70 (from torch)
  Downloading https://download.pytorch.org/whl/cu121/nvidia_cudnn_cu12-9.1.0.70-py3-none-manylinux2014_x86_64.whl (664.8 MB)
    664.8/664.8 MB 2.3 MB/s eta 0:00:00
Collecting nvidia-cublas-cu12==12.1.3.1 (from torch)
  Downloading https://download.pytorch.org/whl/cu121/nvidia_cublas_cu12-12.1.3.1-py3-none-manylinux1_x86_64.whl (410.6 MB)
    410.6/410.6 MB 5.0 MB/s eta 0:00:00
Collecting nvidia-cufft-cu12==11.0.2.54 (from torch)
  Downloading https://download.pytorch.org/whl/cu121/nvidia_cufft_cu12-11.0.2.54-py3-none-manylinux1_x86_64.whl (121.6 MB)
    121.6/121.6 MB 8.5 MB/s eta 0:00:00
Collecting nvidia-curand-cu12==10.3.2.106 (from torch)
  Downloading https://download.pytorch.org/whl/cu121/nvidia_curand_cu12-10.3.2.106-py3-none-manylinux1_x86_64.whl (56.5 MB)
    56.5/56.5 MB 14.8 MB/s eta 0:00:00
Collecting nvidia-cusolver-cu12==11.4.5.107 (from torch)
  Downloading https://download.pytorch.org/whl/cu121/nvidia_cusolver_cu12-11.4.5.107-py3-none-manylinux1_x86_64.whl (124.2 MB)
    124.2/124.2 MB 8.0 MB/s eta 0:00:00
Collecting nvidia-cuspars-cu12==12.1.0.106 (from torch)
  Downloading https://download.pytorch.org/whl/cu121/nvidia_cuspars-cu12-12.1.0.106-py3-none-manylinux1_x86_64.whl (196.0 MB)
    196.0/196.0 MB 5.8 MB/s eta 0:00:00
Collecting nvidia-nccl-cu12==2.21.5 (from torch)
  Downloading https://download.pytorch.org/whl/nvidia_nccl_cu12-2.21.5-py3-none-manylinux2014_x86_64.whl (188.7 MB)
    188.7/188.7 MB 6.8 MB/s eta 0:00:00
Collecting nvidia-nvtx-cu12==12.1.105 (from torch)
  Downloading https://download.pytorch.org/whl/cu121/nvidia_nvtx_cu12-12.1.105-py3-none-manylinux1_x86_64.whl (99 kB)
    99.1/99.1 kB 11.6 MB/s eta 0:00:00
```

Step 2: Data Loading & Label Configuration

```

import json
from datasets import load_dataset
import sys
import os
# sys.path.append(os.path.abspath('.'))
dataset_path = "/content/train_real_tech_cleaned.json"

# 1. Define Labels (Including Soft Skills)
label_list = ["O", "B-SKILL", "I-SKILL", "B-ROLE", "I-ROLE", "B-EDU", "I-EDU", "B-CERT", "I-CERT", "B-SOFT", "I-SOFT"]

id2label = {i: label for i, label in enumerate(label_list)}
label2id = {label: i for i, label in enumerate(label_list)}

print(f"Total labels: {len(label_list)}")

# 2. Load and Split Dataset
dataset = load_dataset('json', data_files='train_real_tech_cleaned.json')
dataset = dataset['train'].train_test_split(test_size=0.1, seed=42)

print(dataset)

```

Total labels: 11

Generating train split: 45911/0 [00:00<00:00, 125364.58 examples/s]

```

DatasetDict({
  train: Dataset({
    features: ['text', 'entities'],
    num_rows: 41319
  })
  test: Dataset({
    features: ['text', 'entities'],
    num_rows: 4592
  })
})

```

▼ Step 3: Tokenization & Label Alignment

```

from transformers import AutoTokenizer

model_checkpoint = "bert-base-cased"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)

def tokenize_and_align_labels(examples):
    tokenized_inputs = tokenizer(
        examples["text"],
        truncation=True,
        max_length=512,
        return_offsets_mapping=True
    )

    labels = []
    for i, entity_list in enumerate(examples["entities"]):
        word_ids = tokenized_inputs.word_ids(batch_index=i)
        offsets = tokenized_inputs["offset_mapping"][i]
        label_ids = [0] * len(word_ids)

        seen_chars = set()
        for ent in entity_list:
            ent_text = ent["text"]
            ent_label = ent["label"]
            start_char = examples["text"][i].find(ent_text)
            if start_char == -1: continue
            end_char = start_char + len(ent_text)

            b_label = label2id.get(f"B-{ent_label}", 0)
            i_label = label2id.get(f"I-{ent_label}", 0)

            first = True
            for idx, (tok_start, tok_end) in enumerate(offsets):
                if tok_start == tok_end: continue # special token
                if tok_start >= start_char and tok_end <= end_char:
                    if first:
                        label_ids[idx] = b_label
                        first = False
                else:

```

```
label_ids[idx] = i_label
seen_chars.add(idx)

for idx, wid in enumerate(word_ids):
    if wid is None:
        label_ids[idx] = -100

labels.append(label_ids)
tokenized_inputs.pop("offset_mapping")
tokenized_inputs["labels"] = labels
return tokenized_inputs

tokenized_datasets = dataset.map(tokenize_and_align_labels, batched=True)
print("✅ Tokenization and Offset Mapping Complete!")
```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
config.json: 100% 570/570 [00:00<00:00, 17.3kB/s]

tokenizer_config.json: 100% 49.0/49.0 [00:00<00:00, 2.28kB/s]

vocab.txt: 213k/? [00:00<00:00, 2.78MB/s]

tokenizer.json: 436k/? [00:00<00:00, 3.45MB/s]

Map: 100% 41319/41319 [00:13<00:00, 3062.40 examples/s]

Map: 100% 4592/4592 [00:01<00:00, 3200.10 examples/s]

✅ Tokenization and Offset Mapping Complete!

```
print("\n--- Verification: Checking Token-Label Alignment ---")
sample = tokenized_datasets["train"][0]
tokens = tokenizer.convert_ids_to_tokens(sample["input_ids"])
print(f"\nSample Text:\n{dataset['train'][0]['text']}\n")
print(f"{'-'*30}")
print(f"{'Token':<20} | {'Label':<15}")
print(f"{'-'*30}")
for tok, lbl_id in zip(tokens, sample["labels"]):
    if lbl_id == -100:
        label_str = 'IGNORED (-100)'
    else:
        label_str = id2label[lbl_id]
    if tok not in ['[PAD]', '[CLS]', '[SEP]'] or lbl_id != -100:
        print(f"{tok:<20} | {label_str:<15}")
```

--- Verification: Checking Token-Label Alignment ---

Sample Text:
Certified Data Scientist (IBM) with strong Problem Solving skills; implemented NLP models in PyTorch.

Token	Label

Certified	B-CERT
Data	I-CERT
Scientist	I-CERT
(	O
IBM	O
)	O
with	O
strong	O
Problem	B-SOFT
Sol	I-SOFT
##ving	I-SOFT
skills	O
;	O
implemented	O
NL	B-SKILL
##P	I-SKILL
models	O

```

in      | 0
P       | B-SKILL
##y     | I-SKILL
##T     | I-SKILL
##or    | I-SKILL
##ch    | I-SKILL
.       | 0

```

Step 4: Model Initialization & Metrics Definition

```

from transformers import AutoModelForTokenClassification, DataCollatorForTokenClassification
import evaluate
import numpy as np

# Initialize Model with ignore_mismatched_sizes
model = AutoModelForTokenClassification.from_pretrained(
    model_checkpoint,
    num_labels=len(label_list),
    id2label=id2label,
    label2id=label2id,
    ignore_mismatched_sizes=True
)

data_collator = DataCollatorForTokenClassification(tokenizer=tokenizer)
sequeval = evaluate.load("sequeval")

def compute_metrics(p):
    predictions, labels = p
    predictions = np.argmax(predictions, axis=2)

    true_predictions = [
        [label_list[p] for (p, l) in zip(prediction, label) if l != -100]
        for prediction, label in zip(predictions, labels)
    ]
    true_labels = [
        [label_list[l] for (p, l) in zip(prediction, label) if l != -100]
        for prediction, label in zip(predictions, labels)
    ]

    results = sequeval.compute(predictions=true_predictions, references=true_labels)
    return {
        "precision": results["overall_precision"],
        "recall": results["overall_recall"],
        "f1": results["overall_f1"],
        "accuracy": results["overall_accuracy"],
    }

```

model.safetensors: 100% 436M/436M [00:03<00:00, 612MB/s]

Loading weights: 100% 197/197 [00:00<00:00, 3615.70it/s]

BertForTokenClassification LOAD REPORT from: bert-base-cased

Key	Status
bert.pooler.dense.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
bert.pooler.dense.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.predictions.bias	UNEXPECTED
classifier.weight	MISSING
classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Downloading builder script: 6.34k/? [00:00<00:00, 619kB/s]

Step 5: Model Training & Evaluation

```

from transformers import TrainingArguments, Trainer, EarlyStoppingCallback

```

```

training_args = TrainingArguments(
    output_dir="./career_compass_results",
    eval_strategy="epoch",          # التقييم في نهاية كل Epoch
    save_strategy="epoch",         # الحفظ برضه في نهاية كل Epoch (لازم يطابق التقييم)
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=5,
    weight_decay=0.01,
    save_total_limit=2,             # يحفظ آخر نسختين بس عشان المساحة
    load_best_model_at_end=True,    # F1 يحمل أحسن نسخة في الآخر بناءً على الـ
    metric_for_best_model="f1",
    report_to="none",              # Logging عشان يشيل أي تحذيرات خاصة بالـ
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_datasets["train"],
    eval_dataset=tokenized_datasets["test"],
    processing_class=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=3)]
)

# السطر ده هو اللي بيبدأ عملية التدريب الفعلية
trainer.train()

```

[12915/12915 38:51, Epoch 5/5]

Epoch	Training Loss	Validation Loss	Precision	Recall	F1	Accuracy
1	0.077623	0.069118	0.921027	0.928206	0.924603	0.973227
2	0.061530	0.064051	0.915886	0.941504	0.928518	0.974912
3	0.053831	0.063463	0.928387	0.943469	0.935867	0.976233
4	0.044553	0.064025	0.932287	0.937967	0.935118	0.977018
5	0.037280	0.068058	0.933307	0.940521	0.936900	0.976376

Writing model shards: 100%

1/1 [00:04<00:00, 4.70s/it]

Writing model shards: 100%

1/1 [00:04<00:00, 4.08s/it]

Writing model shards: 100%

1/1 [00:02<00:00, 2.79s/it]

Writing model shards: 100%

1/1 [00:10<00:00, 10.16s/it]

Writing model shards: 100%

1/1 [00:04<00:00, 4.50s/it]

There were missing keys in the checkpoint model loaded: ['bert.embeddings.LayerNorm.weight', 'bert.embeddings.LayerNorm.bias', 'bert.embeddings.LayerNorm.beta', 'bert.embeddings.LayerNorm.gamma']
 There were unexpected keys in the checkpoint model loaded: ['bert.embeddings.LayerNorm.beta', 'bert.embeddings.LayerNorm.gamma', 'bert.embeddings.LayerNorm.weight', 'bert.embeddings.LayerNorm.bias']
 TrainOutput(global_step=12915, training_loss=0.06115061102382351, metrics={'train_runtime': 2332.4084, 'train_samples_per_second': 88.576, 'train_steps_per_second': 5.537, 'total_flos': 5212079739583080.0, 'train_loss': 0.06115061102382351, 'epoch': 5.0})

Step 6: Model Export & Download

```

from google.colab import files
import os

output_dir = "career_compass_ner_final"

# 1. Save Model & Tokenizer
trainer.save_model(output_dir)
tokenizer.save_pretrained(output_dir)
print(f"✅ Model saved successfully to {output_dir}/")

# 2. Compress Directory
zip_name = f"{output_dir}.zip"
os.system(f"zip -r {zip_name} {output_dir}/")
print(f"✅ Folder zipped successfully!")

# 3. Download to Local Machine
files.download(zip_name)

```

Writing model shards: 100%

1/1 [00:07<00:00, 7.91s/it]

- ✓ Model saved successfully to career_compass_ner_final/
- ✓ Folder zipped successfully!